STUDENT MANAGEMENT SYSTEM

JAVA AND DBMS PROJECT REPORT

CS23333 – OBJECT ORIENTED PROGRAMMING USING JAVA

Submitted by

H.SARAVANASELVAN - 250701663
K.SATHYANARYNAN - 250701670
KRISHVANTH - 250701370
SRIRAMGNANSEKRAN - 250701741

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM
(An Autonomous Institution)

ANNA UNIVERSITY, CHENNAI



NOVEMBER 2026

ACADEMIC YEAR 2026–2027

RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report titled **"STUDENT MANAGEMENT SYSTEM"** is the bonafide work of **"H. SARAVANASELVAN (250701663), K. SATHYANARYNAN (250701670), KRISHVANTH (250701370)**, and **SRIRAM GNANSEKRAN (250701741)**" who carried out the project work under my supervision in partial fulfilment of the requirements for the course Object-Oriented Programming and Database Management Systems during the academic year 2026–2027.

SIGNATURE

PONMANI S

Course Incharge

Object Oriented programming using java

Computer Science and Engineering

SIGNATURE

PARTHASARATHY C

Course Incharge

Database Management System

Computer Science and Engineering

INTERNAL EXAMINER

EXTERNAL EXAMINER

STUDENT DECLARATION

We hereby declare that the project entitled “STUDENT MANAGEMENT SYSTEM” is an original work carried out by us as part of the Object-Oriented Programming and Database Management Systems course. The implementation and descriptions in this report are based on the Java source code and MySQL database structure developed by our team.

Student names and signatures:

1. H. Saravanaselvan _____

2. K. Sathyanarynan _____

3. Krishvanth _____

4. Sriram Gnansekran _____

Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our institution and the Department of Computer Science and Engineering for providing the opportunity and facilities to complete this project. We thank our Head of the Department, project guide, and course faculty for their guidance, suggestions, and support throughout the development process.

We also thank our teammates, friends, and family members for their encouragement. Finally, we acknowledge the developers of Java, MySQL, and MySQL Connector/J, whose technologies were used in this project.

ABSTRACT

The Student Management System is a console-based application developed using Java, JDBC and MySQL. Maintaining student information manually through registers or separate files can be time-consuming and may lead to duplication, errors and difficulty in retrieving academic details. This project provides a centralized database-driven solution for managing student records, academic details, marks, attendance and student accounts.

The system has two roles: Administrator and Student. The administrator can add, view, search, update and delete student records; manage student accounts; add marks; and record attendance. Students can securely log in to view their profile, marks, attendance percentage and academic details, and change their password. Passwords are stored using PBKDF2-based hashing with a random salt. JDBC PreparedStatement is used for parameterized database operations, and transactions are used while creating a student record with related account and academic data.

The application is implemented as a command-line program and does not include a website or graphical user interface. The project demonstrates Java classes, methods, exception handling, JDBC connectivity, SQL CRUD operations, joins, relational constraints and role-based access control.

TABLE OF CONTENTS

CHAPTER	CONTENTS	PAGE NO.
1	INTRODUCTION	7
1.1	Problem Statement	7
1.2	Project Overview	7
1.3	Purpose	7
1.4	Scope	8
2	FUNCTIONALITIES PROVIDED	9
2.1	Functionalities	9
2.2	System Requirements	9
3	SYSTEM FLOW DIAGRAM	10
3.1	Data Flow Diagram	10
3.2	Use Case Diagram	11
3.3	Entity Relationship Diagram	12
4	MODULE DESCRIPTION	13
5	IMPLEMENTATION	14
5.1	Application Entry & Login	14
5.2	Admin Dashboard	16
5.3	Student Dashboard	20
5.4	Database Tables	24
5.5	Program Code	27
6	VERIFICATION & TEST PLAN	42
7	SECURITY, LIMITATIONS & MAINTENANCE	45
8	CONCLUSION & FUTURE WORK	47
9	REFERENCES	48

1 · INTRODUCTION

1.1 PROBLEM STATEMENT

Managing student information manually is a time-consuming and complex process in educational institutions. Maintaining student records, academic details, marks, attendance, and login information in separate registers or files can lead to data duplication, errors, and difficulty in retrieving information.

The proposed Student Management System aims to overcome these challenges by developing a computerized application using Java and MySQL. The system provides a centralized platform for storing, managing, updating, and retrieving student information efficiently.

It allows administrators to manage student records, marks, attendance, and student accounts, while students can securely log in to view their personal details, academic performance, and attendance records. The system also provides secure database connectivity and password management.

The main objective is to reduce manual work, improve data accuracy, ensure secure access, and make student information management faster, easier, and more reliable.

1.2 PROJECT OVERVIEW

The Student Management System is a menu-driven Java console application connected to a MySQL database through JDBC. The application begins with a role selection menu for Admin Login and Student Login. An administrator can maintain student records and related academic data, whereas each student account is linked to a student ID and can access only the corresponding student information.

The system uses separate Java classes for the main application, database connection, password utility, student model, administrator operations and student operations. Database tables store students, user accounts, academic details, marks and attendance.

1.3 PURPOSE

- To maintain student information in a centralized database.
- To reduce manual record maintenance and improve retrieval speed.
- To provide CRUD operations for student records.
- To manage student login accounts and account status.
- To record marks and attendance and allow students to view them.
- To calculate and display attendance percentage.
- To protect passwords using salted PBKDF2 hashing.
- To demonstrate Java OOP concepts and JDBC–MySQL integration.

1.4 SCOPE

The current implementation is a console-based academic prototype. It supports administrator and student login, student record management, account management, marks entry, attendance entry, profile viewing and academic detail viewing. The application requires a local MySQL server and configured database. It does not include a website, mobile application, GUI, automated timetable, fee management, parent portal or cloud deployment.

2 · FUNCTIONALITIES PROVIDED

2.1 FUNCTIONALITIES

- Admin and Student role-based login.
- Automatic creation of a default administrator account when no admin exists.
- Add student with personal and academic information.
- View all student records.
- Search student by ID or name.
- Update selected student details or all details.
- Delete a student record after confirmation.
- View marks of all students using a JOIN with student records.
- Manage student accounts: view, activate, deactivate and reset password.
- Add marks with validation for total and obtained marks.
- Add attendance as PRESENT or ABSENT.
- Student profile view with contact and course information.
- Student marks view filtered by student ID.
- Student attendance history and attendance percentage calculation.
- View academic details including admission year, course, department, current year, semester and CGPA.
- Change student password after verifying the current password.
- Logout and return to the main menu.

2.2 SYSTEM REQUIREMENTS

Hardware Requirements

Component	**Minimum / Suggested Requirement**
Processor	Dual-core processor or above
RAM	4 GB or above

Storage	At least 500 MB free
Display	Terminal / command prompt
Network	Not required for local MySQL setup

Software Requirements

Software	**Purpose**
JDK	Compile and execute Java program
MySQL Server	Relational database storage
MySQL Connector/J	JDBC driver
MySQL Workbench	Database creation and inspection
VS Code / Java IDE	Source code editing
Windows / compatible OS	Execution environment

3 · SYSTEM FLOW DIAGRAM

The system flow diagrams represent the interaction between the administrator, student, Java application processes and the MySQL data stores.

3.1 DATA FLOW DIAGRAM

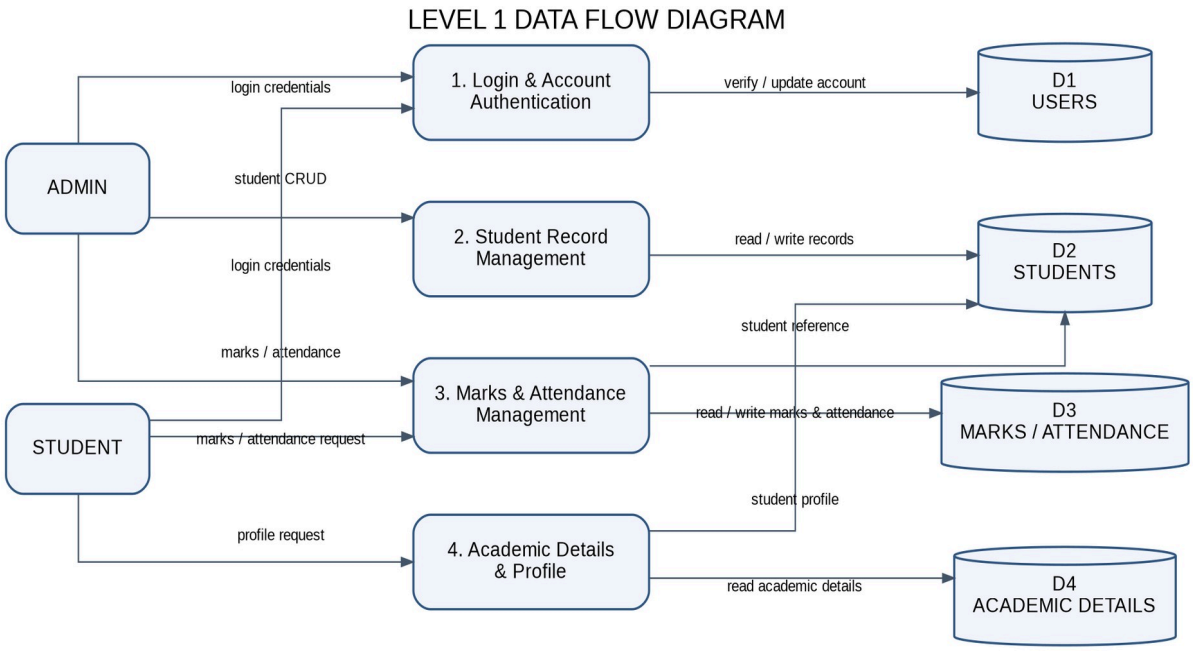


Figure 3.1: Level-1 Data Flow Diagram

The administrator sends login credentials and management actions to the application. The application authenticates users and processes student record, marks, attendance and academic detail requests.

Database operations read or modify the corresponding data stores. Students request their own profile and academic records after successful authentication.

3.2 USE CASE DIAGRAM

Actor	Use Case	Description
Administrator	Login	Authenticates using admin credentials.
Administrator	Add / View / Search Student	Creates and retrieves student records.
Administrator	Update / Delete Student	Modifies or removes student information.
Administrator	Manage Accounts	Views, activates, deactivates or resets student accounts.
Administrator	Add Marks	Records exam marks for a student.
Administrator	Add Attendance	Records PRESENT or ABSENT status.
Student	Login	Authenticates with student username and password.
Student	View Profile	Displays personal and course details.
Student	View Marks	Displays marks belonging to the logged-in student.
Student	View Attendance	Displays attendance history and percentage.
Student	View Academic Details	Displays academic information.
Student	Change Password	Verifies current password and updates the password hash.

Figure 3.2: Use Case Summary (actors: Administrator and Student)

3.3 ENTITY RELATIONSHIP DIAGRAM

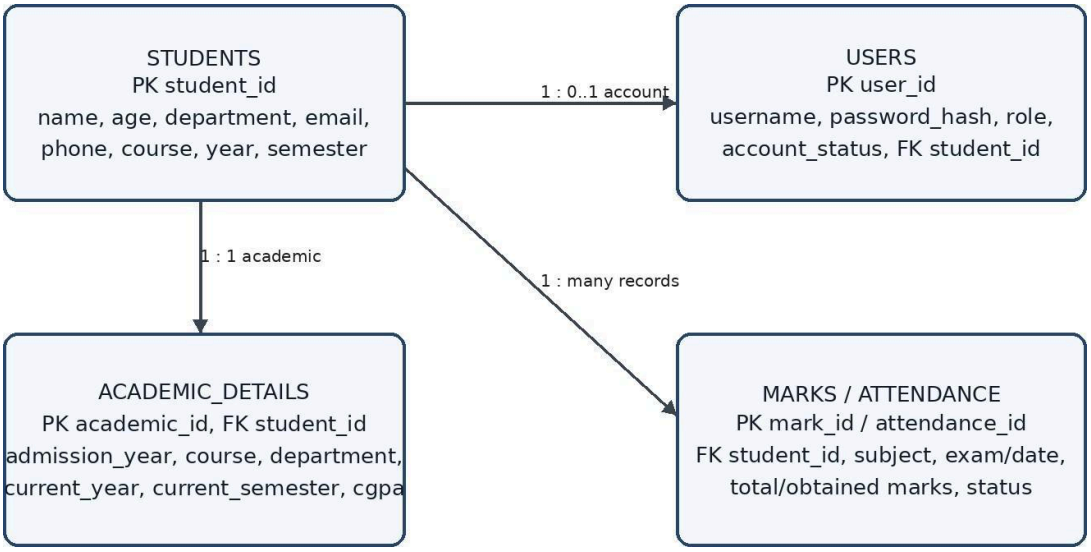


Figure 3.3: Entity Relationship Diagram

The student_id acts as the primary identifier in the students table and is referenced by user accounts, academic details, marks and attendance records. The relationship between students and marks/attendance is one-to-many. A student account is associated with a student record, while the administrator account does not require a student_id.

4 · MODULE DESCRIPTION

MAIN APPLICATION / LOGIN

Main.java displays the top-level menu, reads user choices, creates the initial admin account if needed and authenticates users by username, password and role.

DATABASE CONNECTION

DBConnection.java establishes a connection to the student_management MySQL database using DriverManager and the MySQL JDBC URL.

PASSWORD MANAG

EMENT

PasswordUtil.java generates a random salt and derives a 256-bit hash using PBKDF2WithHmacSHA256. It verifies passwords by deriving a new hash and comparing byte arrays.

ADMIN DASHBOARD

AdminDashboard.java provides the administrator menu and operations for student CRUD, marks, attendance and student account management.

STUDENT DASHBOARD

StudentDashboard.java allows a logged-in student to view their profile, marks, attendance and academic details and to change the password.

STUDENT MODEL / DAO

Student.java represents student attributes. StudentDAO.java contains basic insert and list operations using JDBC.

5 · IMPLEMENTATION

The application is developed in Java and uses JDBC to connect to MySQL. The source files are organized as Main.java, DBConnection.java, PasswordUtil.java, Student.java, StudentDAO.java, AdminDashboard.java and StudentDashboard.java. The project is executed from the terminal; no website or browser is used.

5.1 APPLICATION ENTRY POINT AND DATABASE CONNECTION

Main.java contains the main() method. It initializes the default admin account, displays the main menu and calls the login method based on the selected role. The login method retrieves the account by username, verifies the password hash, checks the role and account status, and opens the appropriate dashboard.

MAIN CLASS CODE (Main.java)

```
import java.sql.*;
import
java.util.Scanner;
public class Main
{
  public static Scanner sc =
  new Scanner(System.in);
  public static void
  main(String[] args) {
    createAd
    min();
    while
    (true) {
      System.out.println("\n===== STUDENT MANAGEMENT SYSTEM
```

```
=====");
System.out.println("1.
Admin Login");
System.out.println("2.
Student Login");
System.out.println("3.
Exit");

System.out.print("En
ter choice: "); int
choice = readInt();

if (choice == 1)
  {
  login("ADMI
  N");
} else if
  (choice == 2)
  {
  login("STUD
  ENT");
} else if (choice == 3) {
  System.out.println("T
  hank you!"); break;
```

```
    } else {
      System.out.println("Invalid choice.");
    }
  }
}

public static int
  readInt() {
  while (true) {
    try {
      return Integer.parseInt(sc.nextLine());
    } catch (NumberFormatException e)
      { System.out.print("Enter a valid
      number: ");
    }
  }
}

static void createAdmin() {
  try (Connection con =
    DBConnection.getConnection()) {
    String sql =
      "SELECT COUNT(*) FROM users WHERE role='ADMIN'";

    Statement st =
```

```
    con.createStatement();
    ResultSet rs =
    st.executeQuery(sql);

    rs.next();
    if (rs.getInt(1) ==
      0) { String
      insert =
        "INSERT INTO users(username,password_hash,role)
      VALUES(?,?,?)"; PreparedStatement ps =
      con.prepareStatement(insert);
      ps.setString(1, "admin");
      ps.setString(2,
      PasswordUtil.hashPassword("Admin@123"));
      ps.setString(3, "ADMIN");

      ps.executeUpdate();

      System.out.println("Default Admin Created!");
    }

  } catch (Exception e) {
    System.out.println("Admin Setup Error: " + e.getMessage());
  }
}

static void login(String requestedRole) {
```

```
try (Connection con =
  DBConnection.getConnection()) {
  System.out.print("Enter Username: ");
  String username = sc.nextLine();

  System.out.print("Enter
  Password: "); String
  password = sc.nextLine()
  String sql = "SELECT * FROM users WHERE username=?";

  PreparedStatement ps =
  con.prepareStatement(sql);
  ps.setString(1, username);
  ResultSet rs =
  ps.executeQuery(); if
  (!rs.next()) {
    System.out.println("Username not found.");
    return;
  }

  String storedPassword =
  rs.getString("password_hash"); String
  role = rs.getString("role");
  String status =
  rs.getString("account_status");
  int studentId =
  rs.getInt("student_id");
```

```
if (!PasswordUtil.verifyPassword(password,
  storedPassword)) {
  System.out.println("Incorrect password.");
  return;
};

if
  (!role.equals(requestedR
  ole)) {
  System.out.println("Acce
  ss denied."); return;
}

if (!"ACTIVE".equals(status)) {
  System.out.println("Your account
  is inactive."); return;
}
System.out.println("Login
Successful!"); rs.close();
ps.close();

if (role.equals("ADMIN")) {
  AdminDashboard.show()
  ;
} else {
  StudentDashboard.show(studentId);
```

```
        }


      } catch (Exception e) {
        System.out.println("Login Error: " + e.getMessage());
      }
    }
}
mport java.sql.DriverManager;


public class DBConnection {
  public static Connection
    getConnection() { try {
      String url =
      "jdbc:mysql://localhost:3306/student_manage
      ment"; String user = "root";
      String password = "YOUR_MYSQL_PASSWORD";
      Connection con = DriverManager.getConnection(url, user,
      password); System.out.println("Database Connected
      Successfully!");
      return con;


    } catch
      (Exception e) {
      System.out.pri
      ntln(e); return
      null;
    }
```

```
  }
}
```

5.2 ADMIN DASHBOARD

The administrator dashboard contains ten menu choices: add student, view all students, search student, update student, delete student, view all marks, manage student accounts, add marks, add attendance and logout. Student insertion is performed in a transaction so the student, user account and academic detail records are committed together or rolled back if an operation fails.

PASSWORD UTILITY CODE (PasswordUtil.java)

```
import
java.security.MessageDi
gest;                   import
java.security.SecureRan
dom;                    import
java.util.Base64;
import
javax.crypto.SecretKeyFac
tory;                    import
javax.crypto.spec.PBEKey
Spec;


public class PasswordUtil {
  public static String hashPassword(String
    password) throws Exception { byte[] salt = new
    byte[16];
    new SecureRandom().nextBytes(salt);


    PBEKeySpec spec = new
      PBEKeySpec(
      password.toCharArray(), salt,
      65536, 256
```

```
    );

    SecretKeyFactory factory =
      SecretKeyFactory.getInstance("PBKDF2WithH
      macSHA256");

    byte[] hash = factory.generateSecret(spec).getEncoded();

    return
        Base64.getEncoder().encodeToStri
        ng(salt) + ":" +
        Base64.getEncoder().encodeToStri
        ng(hash);
  }

  public static boolean
    verifyPassword( String
    password, String
    storedPassword
  ) throws Exception {

    String[] parts = storedPassword.split(":");

    byte[] salt =
    Base64.getDecoder().decode(parts[0
    ]); byte[] oldHash =
```

```
Base64.getDecoder().decode(parts[1
]);

PBEKeySpec spec = new
  PBEKeySpec(
  password.toCharArray(), salt,
  65536, 256
);

SecretKeyFactory factory =
  SecretKeyFactory.getInstance("PBKDF2WithH
  macSHA256");
```

```
        return MessageDigest.isEqual(oldHash, newHash);
    }
}
```

STUDENT MODEL CODE (Student.java)

```
public class
    Student { int
    id;
    String
    name;
    int age;
    String
    department;
    String email;
    String
    phone;

    public Student(int id, String name, int age, String department,
        String email, String phone) { this.id = id;
        this.name =
        name; this.age =
        age;
        this.department =
        department; this.email =
        email;
        this.phone = phone;
    }
}
```

5.2 ADMIN DASHBOARD · CONTINUED

ADMIN DASHBOARD CODE (AdminDashboard.java)

```
import
java.sql.*;
public class
AdminDashboard {
static void show() {
while (true) {
System.out.println("\n===== ADMIN
DASHBOARD =====");
System.out.println("1. Add Student");
System.out.println("2. View All
Students"); System.out.println("3.
Search Student");
System.out.println("4. Update
Student"); System.out.println("5.
Delete Student");
System.out.println("6. View All Marks");
System.out.println("7. Manage Student
Accounts"); System.out.println("8. Add
Student Marks"); System.out.println("9.
Add Student Attendance");
System.out.println("10. Logout");

System.out.print("Enter
your choice: "); int choice =
Main.readInt();
```

```
    switch (choice) {
      case 1: addStudent();
      break; case 2:
      viewStudents();
      break; case 3:
      searchStudent();
      break; case 4:
      updateStudent();
      break; case 5:
      deleteStudent();
      break; case 6:
      viewMarks(); break;
      case 7:
      manageAccounts();
      break; case 8:
      addMarks(); break;
      case 9:
      addAttendance();
      break; case 10:
        System.out.println("Admin
        Logged Out!"); return;
      default:
        System.out.println("Invalid Choice!");
    }
  }
}
```

```
static void addStudent() {
  try (Connection con =
    DBConnection.getConnection()) {
    con.setAutoCommit(false);


    try {
      String sql = "INSERT INTO
      students(name,age,department,email,phone,course,year,semester)
      VALUES(?,?,?,?,?,?,?,?)"; PreparedStatement ps = con.prepareStatement(sql,
      Statement.RETURN_GENERATED_KEYS);
```

```
System.out.print("Enter
Name: "); ps.setString(1,
Main.sc.nextLine());

System.out.print("Ente
r Age: "); ps.setInt(2,
Main.readInt());

System.out.print("Enter
Department: "); String
department =
Main.sc.nextLine();
ps.setString(3,
department);

System.out.print("Enter
Email: "); ps.setString(4,
Main.sc.nextLine());

System.out.print("Enter
Phone: "); ps.setString(5,
Main.sc.nextLine());

System.out.print("Ente
r Course: "); String
course =
Main.sc.nextLine();
```

```
ps.setString(6, course);

System.out.print("Ente
r Year: "); int year =
Main.readInt();
ps.setInt(7, year);

System.out.print("Enter
Semester: "); int
semester =
Main.readInt();
ps.setInt(8, semester);

ps.executeUpdate();

ResultSet keys =
ps.getGeneratedKeys();
keys.next();
int studentId = keys.getInt(1);

System.out.print("Enter
Username: "); String
username =
Main.sc.nextLine();

System.out.print("Enter
Password: "); String
```

```
        password =
        Main.sc.nextLine();

        String userSql = "INSERT INTO
        users(username,password_hash,role,student_id) VALUES(?,?,?,?)";
        PreparedStatement userPs = con.prepareStatement(userSql);

        userPs.setString(1, username);
        userPs.setString(2,
        PasswordUtil.hashPassword(password));
        userPs.setString(3, "STUDENT");
        userPs.setInt(4,
        studentId);
        userPs.executeUp
        date();

        String academicSql = "INSERT INTO
academic_details(student_id,admission_year,course,department,current_year,current_
semester) VALUES(?,?,?,?,?,?)";
```

```
PreparedStatement academicPs = con.prepareStatement(academicSql);

academicPs.setInt(1, studentId);
academicPs.setInt(2,
java.time.Year.now().getValue());
academicPs.setString(3, course);
academicPs.setString(4,
department); academicPs.setInt(5,
year);
academicPs.setInt(6,
semester);
academicPs.execute
Update();

con.commit();

System.out.println("Student Added
Successfully!");
System.out.println("Student ID: " +
studentId);

keys.close();
ps.close();
userPs.close()
;
academicPs.cl
```

```
        ose();

      } catch
        (Exception e)
        {
        con.rollback(
        ); throw e;
      }

    } catch (Exception e) {
      System.out.println("Error: " +
      e.getMessage());
    }
}

static void viewStudents() {
    String sql = "SELECT * FROM students";

    try (Connection con =
        DBConnection.getConnection();
        PreparedStatement ps =
        con.prepareStatement(sql); ResultSet
        rs = ps.executeQuery()) {
      System.out.println("\n===== ALL
      STUDENTS ====="); while (rs.next()) {
        System.out.println("\nStudent ID: " + rs.getInt("student_id"));
```

```
    System.out.println("Name: " +
    rs.getString("name"));
    System.out.println("Age: " + rs.getInt("age"));
    System.out.println("Department: " +
    rs.getString("department"));
    System.out.println("Email: " +
    rs.getString("email"));
    System.out.println("Phone: " +
    rs.getString("phone"));
    System.out.println("Course: " +
    rs.getString("course"));
    System.out.println("Year: " +
    rs.getInt("year"));
    System.out.println("Semester: " +
    rs.getInt("semester"));
  }
```

```
  } catch (Exception e) {
    System.out.println("Error: " +
    e.getMessage());
  }
}

static void searchStudent() {
  try (Connection con = DBConnection.getConnection()) {

    System.out.println("\n1.
    Search By ID");
    System.out.println("2. Search
    By Name");

    System.out.print("Enter
    Choice: "); int choice =
    Main.readInt();

    String sql;

    if (choice == 1) {
      sql = "SELECT * FROM students WHERE student_id=?";
    } else if (choice == 2) {
      sql = "SELECT * FROM students WHERE name LIKE ?";
    } else {
      System.out.println("Invali
      d Choice!"); return;
```

```
}
PreparedStatement ps =
con.prepareStatement(sql); if
(choice == 1) {
  System.out.print("Enter
  Student ID: "); ps.setInt(1,
  Main.readInt());
} else {
  System.out.print("Enter Student
  Name: "); ps.setString(1, "%" +
  Main.sc.nextLine() + "%");
}
ResultSet rs =
ps.executeQuery(); if
(rs.next()) {
  do {
    System.out.println("\nStudent ID: " +
    rs.getInt("student_id"));
    System.out.println("Name: " +
    rs.getString("name"));
    System.out.println("Age: " +
    rs.getInt("age"));
    System.out.println("Department: " +
    rs.getString("department"));
    System.out.println("Email: " +
    rs.getString("email"));
    System.out.println("Phone: " +
    rs.getString("phone"));
```

```
      System.out.println("Course: " +
      rs.getString("course"));
      System.out.println("Year: " +
      rs.getInt("year"));
      System.out.println("Semester: " +
      rs.getInt("semester"));
    } while (rs.next());
  } else {
    System.out.println("Student Not Found!");
  }
```

```
    rs.close();
    ps.close();

  } catch (Exception e) {
    System.out.println("Error: " +
    e.getMessage());
  }
}

static void updateStudent() {
  try (Connection con = DBConnection.getConnection()) {

    System.out.print("Enter
    Student ID: "); int id =
    Main.readInt();

    String check = "SELECT * FROM students
    WHERE student_id=?"; PreparedStatement
    ps = con.prepareStatement(check);
    ps.setInt(1, id);
    ResultSet rs =
    ps.executeQuery(); if
    (!rs.next()) {
      System.out.println("Student Not Found!");
      rs.close();
      ps.close
```

```
  ();
  return;
}

System.out.println("\n===== STUDENT
DETAILS ====="); System.out.println("Name: "
+ rs.getString("name"));
System.out.println("Age: " + rs.getInt("age"));
System.out.println("Department: " +
rs.getString("department"));
System.out.println("Email: " +
rs.getString("email"));
System.out.println("Phone: " +
rs.getString("phone"));
System.out.println("Course: " +
rs.getString("course"));
System.out.println("Year: " + rs.getInt("year"));
System.out.println("Semester: " +
rs.getInt("semester"));

rs.close();
ps.close();

System.out.println("\n===== WHICH DETAIL DO YOU WANT TO UPDATE? =====");
System.out.println("1. Name");
System.out.println("2. Age");
System.out.println("3.
```

```
Department");
System.out.println("4. Email");
System.out.println("5.
Phone");
System.out.println("6.
Course");
System.out.println("7. Year");
System.out.println("8.
Semester");
System.out.println("9. Update
All Details");
System.out.println("10. Back");
```

```
System.out.print("Enter
your choice: "); int choice =
Main.readInt();

if (choice == 10) return;

String name =
null; String
department =
null; String email
= null; String
phone = null;
String course =
null;
int age
= 0; int
year =
0;
int semester = 0;

if (choice == 9) {
  System.out.print("Enter
  New Name: "); name =
  Main.sc.nextLine();

  System.out.print("Enter
  New Age: "); age =
```

```
Main.readInt();

System.out.print("Enter New
Department: "); department =
Main.sc.nextLine();

System.out.print("Enter
New Email: "); email =
Main.sc.nextLine();

System.out.print("Enter New
Phone: "); phone =
Main.sc.nextLine();

System.out.print("Enter New
Course: "); course =
Main.sc.nextLine();

System.out.print("Enter
New Year: "); year =
Main.readInt();

System.out.print("Enter New
Semester: "); semester =
Main.readInt();
con.setAutoCom
```

```
        mit(false); try {
          String sql = "UPDATE students SET
          name=?,age=?,department=?,email=?,phone=?,course=?,year=?,semester=?
WHERE student_id=?";
          PreparedStatement update = con.prepareStatement(sql);


          update.setString(1,
          name);
          update.setInt(2, age);
          update.setString(3,
          department);
          update.setString(4,
          email);
```

```
    update.setString(
    5, phone);
    update.setString(
    6, course);
    update.setInt(7,
    year);
    update.setInt(8,
    semester);
    update.setInt(9,
    id);

    update.executeUpda
    te(); update.close();
    updateAcademic(con, id, department,
    course, year, semester); con.commit();
    System.out.println("Student Details Updated Successfully!");

  } catch
    (Exception e)
    {
    con.rollback()
    ; throw e;
  }

  return;
}
```

```
String
column;
String value;


switch
  (choice) {
  case 1:
    column = "name";
    System.out.print("Enter
    New Name: "); value =
    Main.sc.nextLine();
    brea
  k;
  case
  2:
    column = "age";
    System.out.print("Enter
    New Age: "); value =
    Main.sc.nextLine();
    brea
  k;
  case
  3:
    column = "department";
    System.out.print("Enter New
    Department: "); value =
    Main.sc.nextLine();
    brea
```

```
k;
case
4:
  column = "email";
  System.out.print("Enter
  New Email: "); value =
  Main.sc.nextLine();
  brea
k;
case
5:
  column = "phone";
  System.out.print("Enter
  New Phone: "); value =
  Main.sc.nextLine();
  brea
k;
case
6:
  column = "course";
```

```
    System.out.print("Enter
    New Course: "); value =
    Main.sc.nextLine();
    brea
  k;
  case
  7:
    column = "year";
    System.out.print("Enter
    New Year: "); value =
    Main.sc.nextLine();
    brea
  k;
  case
  8:
    column = "semester";
    System.out.print("Enter New
    Semester: "); value =
    Main.sc.nextLine();
    break;
  default:
    System.out.println("Invalid
    Choice!"); return;
}

String sql = "UPDATE students SET " + column + "=?
WHERE student_id=?"; PreparedStatement update =
con.prepareStatement(sql);
```

```
if (choice == 2 || choice == 7 ||
  choice == 8) { update.setInt(1,
  Integer.parseInt(value));
} else {
  update.setString(1, value);
}

update.setInt(2, id);
update.executeUpdate()
; update.close();

if (choice == 3 || choice == 6 || choice == 7 || choice == 8) {
  String getSql = "SELECT department,course,year,semester FROM students
  WHERE student_id=?"; PreparedStatement get =
  con.prepareStatement(getSql);
  get.setInt(1, id);
  ResultSet data =
  get.executeQuery(); if
  (data.next()) {
    updateAcade
      mic( con,
      id,
      data.getString("dep
      artment"),
      data.getString("cour
      se"),
      data.getInt("year"),
```

```
        data.getInt("semest
        er")
      );
    }

    data.close();
    get.close();
}
```

```
      System.out.println("Student Details Updated Successfully!");


    } catch (Exception e) {
      System.out.println("Error: " +
      e.getMessage());
    }
  }

  static void updateAcademic(Connection con, int
                id, String department, String course,
                int year, int semester)
                throws SQLException {


    String sql = "UPDATE academic_details SET
department=?,course=?,current_year=?,current_semester=? WHERE student_id=?";
    PreparedStatement ps = con.prepareStatement(sql);


    ps.setString(1,
    department);
    ps.setString(2,
    course);
    ps.setInt(3, year);
    ps.setInt(4,
    semester);
    ps.setInt(5, id);


    ps.executeUpdat
```

```
  e(); ps.close();
}

static void deleteStudent() {
  try (Connection con = DBConnection.getConnection()) {

    System.out.print("Enter Student
    ID to Delete: "); int id =
    Main.readInt();

    System.out.print("Are you sure? Enter
    Y to confirm: "); String confirm =
    Main.sc.nextLine();

    if
      (!confirm.equalsIgnoreCase("Y
      ")) { System.out.println("Delete
      Cancelled!"); return;
    }

    String sql = "DELETE FROM students
    WHERE student_id=?";
    PreparedStatement ps =
    con.prepareStatement(sql); ps.setInt(1,
    id);
    int result =
    ps.executeUpdate();
```

```
if (result > 0) {
  System.out.println("Student Deleted Successfully!");
} else {
  System.out.println("Student Not Found!");
}
```

```
    ps.close();

  } catch (Exception e) {
    System.out.println("Error: " +
    e.getMessage());
  }
}

static void viewMarks() {
  String sql = "SELECT s.name,m.* FROM marks m JOIN students s ON
  m.student_id=s.student_id";

  try (Connection con =
    DBConnection.getConnection();
    PreparedStatement ps =
    con.prepareStatement(sql); ResultSet
    rs = ps.executeQuery()) {
    System.out.println("\n===== ALL STUDENT
    MARKS ====="); while (rs.next()) {
      System.out.println("\nStudent: " + rs.getString("name"));
      System.out.println("Student ID: " +
      rs.getInt("student_id")); System.out.println("Subject:
      " + rs.getString("subject")); System.out.println("Total
      Marks: " + rs.getInt("total_marks"));
      System.out.println("Obtained Marks: " +
      rs.getInt("obtained_marks"));
      System.out.println("Exam Type: " +
```

```
        rs.getString("exam_type")); System.out.println("Exam
        Date: " + rs.getDate("exam_date"));
      }

    } catch (Exception e) {
      System.out.println("Error: " +
      e.getMessage());
    }
}

static void
  manageAccounts() {
  while (true) {
    System.out.println("\n===== MANAGE STUDENT
    ACCOUNTS ====="); System.out.println("1. View
    Accounts");
    System.out.println("2. Activate
    Account");
    System.out.println("3.
    Deactivate Account");
    System.out.println("4. Reset
    Password"); Syst
```

6 ECURITY, LIMITATIONS & MAINTENANCE

6.1 SECURITY FEATURES

- Passwords are not stored as plain text; PBKDF2WithHmacSHA256 with random salt is used.
- Password comparison uses MessageDigest.isEqual().
- PreparedStatement is used for parameterized queries to reduce SQL injection risk.
- Role checks distinguish ADMIN and STUDENT accounts.
- Account status can prevent inactive students from logging in.
- Student dashboard queries are filtered using the authenticated student ID.

6.2 LIMITATIONS

- The interface is command-line based and has no website or graphical screens.
- The DB credentials are configured in source code in the current version and should be moved to environment variables or a protected configuration file.
- Some database operations in the source use manual resource closing; try-with-resources can be applied consistently.
- Email and phone validation can be strengthened.
- No export to PDF/Excel, notifications, timetable or fee module is included.
- The system is a local academic prototype and is not deployed for institution-wide use.

6.3 FUTURE MAINTENANCE

- Move database credentials to a secure external configuration.
- Add stronger input validation and centralized exception handling.
- Add audit logs for admin changes.
- Add report generation and CSV/PDF export.
- Develop a Java Swing GUI or web front end only if required in a future version.
- Add backup, restore and database migration procedures.

7 ECURITY, LIMITATIONS & MAINTENANCE

7.1 SECURITY FEATURES

- Passwords are not stored as plain text; PBKDF2WithHmacSHA256 with random salt is used.
- Password comparison uses MessageDigest.isEqual().
- PreparedStatement is used for parameterized queries to reduce SQL injection risk.
- Role checks distinguish ADMIN and STUDENT accounts.
- Account status can prevent inactive students from logging in.
- Student dashboard queries are filtered using the authenticated student ID.

7.2 LIMITATIONS

- The interface is command-line based and has no website or graphical screens.
- The DB credentials are configured in source code in the current version and should be moved to environment variables or a protected configuration file.
- Some database operations in the source use manual resource closing; try-with-resources can be applied consistently.
- Email and phone validation can be strengthened.
- No export to PDF/Excel, notifications, timetable or fee module is included.
- The system is a local academic prototype and is not deployed for institution-wide use.

7.3 FUTURE MAINTENANCE

- Move database credentials to a secure external configuration.
- Add stronger input validation and centralized exception handling.
- Add audit logs for admin changes.
- Add report generation and CSV/PDF export.
- Develop a Java Swing GUI or web front end only if required in a future version.
- Add backup, restore and database migration procedures.

8 ECURITY, LIMITATIONS & MAINTENANCE

8.1 SECURITY FEATURES

- Passwords are not stored as plain text; PBKDF2WithHmacSHA256 with random salt is used.
- Password comparison uses MessageDigest.isEqual().
- PreparedStatement is used for parameterized queries to reduce SQL injection risk.
- Role checks distinguish ADMIN and STUDENT accounts.
- Account status can prevent inactive students from logging in.
- Student dashboard queries are filtered using the authenticated student ID.

8.2 LIMITATIONS

- The interface is command-line based and has no website or graphical screens.
- The DB credentials are configured in source code in the current version and should be moved to environment variables or a protected configuration file.
- Some database operations in the source use manual resource closing; try-with-resources can be applied consistently.
- Email and phone validation can be strengthened.
- No export to PDF/Excel, notifications, timetable or fee module is included.
- The system is a local academic prototype and is not deployed for institution-wide use.

8.3 FUTURE MAINTENANCE

- Move database credentials to a secure external configuration.
- Add stronger input validation and centralized exception handling.
- Add audit logs for admin changes.
- Add report generation and CSV/PDF export.
- Develop a Java Swing GUI or web front end only if required in a future version.
- Add backup, restore and database migration procedures.

9 ECURITY, LIMITATIONS & MAINTENANCE

9.1 SECURITY FEATURES

- Passwords are not stored as plain text; PBKDF2WithHmacSHA256 with random salt is used.
- Password comparison uses MessageDigest.isEqual().
- PreparedStatement is used for parameterized queries to reduce SQL injection risk.
- Role checks distinguish ADMIN and STUDENT accounts.
- Account status can prevent inactive students from logging in.
- Student dashboard queries are filtered using the authenticated student ID.

9.2 LIMITATIONS

- The interface is command-line based and has no website or graphical screens.
- The DB credentials are configured in source code in the current version and should be moved to environment variables or a protected configuration file.
- Some database operations in the source use manual resource closing; try-with-resources can be applied consistently.
- Email and phone validation can be strengthened.
- No export to PDF/Excel, notifications, timetable or fee module is included.
- The system is a local academic prototype and is not deployed for institution-wide use.

9.3 FUTURE MAINTENANCE

- Move database credentials to a secure external configuration.
- Add stronger input validation and centralized exception handling.
- Add audit logs for admin changes.
- Add report generation and CSV/PDF export.
- Develop a Java Swing GUI or web front end only if required in a future version.
- Add backup, restore and database migration procedures.

```
System.out.println("Status: " + status);

            ps.close();

        } catch (Exception e) {
            System.out.println("Error: " +
            e.getMessage());
        }
    }
}
```

```
rintln("5. Back");

        System.out.print("Enter
        Choice: "); int choice =
        Main.readInt();

        if (choice == 5) return;
        try (Connection con =
          DBConnection.getConnection()) { if
          (choice == 1) {
            String sql = "SELECT user_id,username,student_id,account_status FROM users
            WHERE role='STUDENT'"; PreparedStatement ps = con.prepareStatement(sql);
            ResultSet rs = ps.executeQuery();
```

```
    while (rs.next()) {
      System.out.println("\nUser ID: " +
      rs.getInt("user_id")); System.out.println("Username: "
      + rs.getString("username"));
      System.out.println("Student ID: " +
      rs.getInt("student_id")); System.out.println("Status: "
      + rs.getString("account_status"));
    }

    rs.close();
    ps.close();

  } else if (choice == 2 ||
    choice == 3) {
    System.out.print("Enter
    Student ID: "); int id =
    Main.readInt();

    String status = choice == 2 ? "ACTIVE" : "INACTIVE";
    String sql = "UPDATE users SET account_status=? WHERE student_id=? AND
    role='STUDENT'";

    PreparedStatement ps =
    con.prepareStatement(sql);
    ps.setString(1, status);
    ps.setInt(2, id);
    int result =
```

```
    ps.executeUpdate();
    if (result > 0) {
      System.out.println("Account " + status + " Successfully!");
    } else {
      System.out.println("Account Not Found!");
    }

    ps.close();

  } else if (choice == 4) {
    System.out.print("Enter
    Student ID: "); int id =
    Main.readInt();

    System.out.print("Enter New
    Password: "); String password
    = Main.sc.nextLine();

    if (password.length() < 8) {
      System.out.println("Password must contain at
      least 8 characters!"); continue;
    }

    String sql = "UPDATE users SET password_hash=? WHERE student_id=?
    AND role='STUDENT'"; PreparedStatement ps = con.prepareStatement(sql);

    ps.setString(1,
```

```
PasswordUtil.hashPassword(password));
ps.setInt(2, id);

int result = ps.executeUpdate();
```

```
            if (result > 0) {
              System.out.println("Password Reset Successfully!");
            } else {
              System.out.println("Account Not Found!");
            }

            ps.close();

          } else {
            System.out.println("Invalid Choice!");
          }

        } catch (Exception e) {
          System.out.println("Error: " +
          e.getMessage());
        }
      }
    }

    static void addMarks() {
      try (Connection con = DBConnection.getConnection()) {

        System.out.print("Enter
        Student ID: "); int id =
        Main.readInt();
```

```
System.out.print("Enter
Subject: "); String
subject =
Main.sc.nextLine();

System.out.print("Enter
Total Marks: "); int total =
Main.readInt();

System.out.print("Enter
Obtained Marks: "); int
obtained = Main.readInt();

if (total <= 0 || obtained < 0 ||
  obtained > total) {
  System.out.println("Invalid
  Marks!");
  return;
}

System.out.print("Enter
Exam Type: "); String exam
= Main.sc.nextLine();

System.out.print("Enter Exam Date
(YYYY-MM-DD): "); String date =
Main.sc.nextLine();
```

```
        String sql = "INSERT INTO
marks(student_id,subject,total_marks,obtained_marks,exam_type,exam_date)
VALUES(?,?,?,?,?,?)";
        PreparedStatement ps = con.prepareStatement(sql);


        ps.setInt(1, id);
        ps.setString(2,
        subject);
        ps.setInt(3,
        total);
```

```
        PasswordUtil.hashPassword(newPassword));
        ups.setInt(2, id);

        ups.executeUpdate();

        System.out.println("Password Changed Successfully!");

      } catch (Exception e) {
        System.out.println(e.getMessa
        ge());
      }
    }
}
```

```
    ps.setInt(4,
    obtained);
    ps.setString(5,
    exam);
    ps.setDate(6,
    java.sql.Date.valueOf(date));
    ps.executeUpdate();
    System.out.println("Marks Added
    Successfully!"); ps.close();
  } catch (Exception e) {
    System.out.println("Error: " +
    e.getMessage());
  }
}

static void addAttendance() {
  try (Connection con = DBConnection.getConnection()) {

    System.out.print("Enter
    Student ID: "); int id =
    Main.readInt();

    System.out.print("Enter
    Subject: "); String
    subject =
    Main.sc.nextLine();
```

```
System.out.print("Enter Date
(YYYY-MM-DD): "); String date =
Main.sc.nextLine();

System.out.println("\n===== ATTENDANCE
STATUS ====="); System.out.println("1.
PRESENT");
System.out.println("2. ABSENT");

System.out.print("Enter
Choice (1 or 2): "); int choice
= Main.readInt();

String status;

if (choice == 1)
  { status =
  "PRESENT";
} else if (choice
  == 2) { status =
  "ABSENT";
} else {
  System.out.println("Invalid Choice!
  Enter 1 or 2."); return;
}

String sql = "INSERT INTO
```

```
attendance(student_id,subject,attendance_date,status) VALUES(?,?,?,?)";
PreparedStatement ps = con.prepareStatement(sql);


ps.setInt(1, id);
ps.setString(2,
subject);
ps.setDate(3, java.sql.Date.valueOf(date));
```

```
ps.setString(4,
status);
ps.executeUpd
ate();
System.out.println("Attendance Added
Successfully!");
```

5.3 STUDENT DASHBOARD

After successful student login, StudentDashboard.show(studentId) displays the student menu. All student queries use the authenticated student ID, which prevents the student menu from displaying another student's records through ordinary dashboard operations. Attendance percentage is calculated as present_count × 100 / total_attendance_records.

STUDENT DASHBOARD CODE (StudentDashboard.java)

```
import java.sql.*;


public class StudentDashboard {


  public static void show(int
    studentId) { while (true) {
      System.out.println("\n========== STUDENT DASHBOARD ==========");
      System.out.println("1. View My
      Profile"); System.out.println("2. View
      My Marks"); System.out.println("3.
      View My Attendance");
      System.out.println("4. View Academic
      Details"); System.out.println("5.
      Change Password");
      System.out.println("6. Logout");
      System.out.print("Ente
      r choice: "); int choice =
      Main.readInt(); switch
      (choice) {
        case 1: viewProfile(studentId); break;
        case 2: viewMarks(studentId); break;
        case 3:
```

```
        viewAttendance(studentId);
        break; case 4:
        viewAcademic(studentId);
        break; case 5:
        changePassword(studentId);
        break; case 6:
          System.out.println("Student
          Logged Out."); return;
        default:
          System.out.println("Invalid Choice.");
      }
    }
}

static void viewProfile(int id) {

  String sql = "SELECT * FROM students
  WHERE student_id=?"; try (Connection
  con = DBConnection.getConnection()) {
    PreparedStatement ps =
    con.prepareStatement(sql);
    ps.setInt(1, id);
```

```
  ResultSet rs =
  ps.executeQuery(); if
  (rs.next()) {
    System.out.println("\n========== MY
    PROFILE ==========");
    System.out.println("ID: " +
    rs.getInt("student_id"));
    System.out.println("Name: " +
    rs.getString("name"));
    System.out.println("Age: " + rs.getInt("age"));
    System.out.println("Department: " +
    rs.getString("department"));
    System.out.println("Email: " +
    rs.getString("email"));
    System.out.println("Phone: " +
    rs.getString("phone"));
    System.out.println("Course: " +
    rs.getString("course"));
    System.out.println("Year: " +
    rs.getInt("year"));
    System.out.println("Semester: " +
    rs.getInt("semester"));
  } else {
    System.out.println("Profile Not Found.");
  }


} catch (Exception e) {
```

```
    System.out.println(e.getMessa
    ge());
  }
}

static void viewMarks(int id) {

  String sql = "SELECT * FROM marks
  WHERE student_id=?"; try (Connection
  con = DBConnection.getConnection()) {
    PreparedStatement ps =
    con.prepareStatement(sql);
    ps.setInt(1, id);

    ResultSet rs = ps.executeQuery();
    System.out.println("\n========== MY
    MARKS ==========="); boolean found =
    false;
    while
      (rs.next()) {
      found = true;
      System.out.println("Subject: " +
      rs.getString("subject"));
      System.out.println("Exam: " +
      rs.getString("exam_type"));
      System.out.println("Total: " +
      rs.getInt("total_marks"));
```

```
  System.out.println("Obtained: " +
  rs.getInt("obtained_marks"));
  System.out.println("      ");
}
```

```
        System.out.println("No marks found.");
      }


    } catch (Exception e) {
      System.out.println(e.getMessa
      ge());
    }
}
static void
  viewAttendance(int id)
  { String sql =
    "SELECT * FROM attendance WHERE student_id=? ORDER
  BY attendance_date DESC"; try (Connection con =
  DBConnection.getConnection()) {
    PreparedStatement ps =
    con.prepareStatement(sql);
    ps.setInt(1, id);
    ResultSet rs =
    ps.executeQuery(); int
    total = 0;
    int present = 0;


    System.out.println("\n========== MY ATTENDANCE ==========");


    while
      (rs.next()) {
      total++;
```

```
    String status = rs.getString("status");

    if
      (status.equals("PRES
      ENT")) { present++;
    }

    System.out.println(
      rs.getString("subject") + "
      | " +
      rs.getDate("attendance_
      date") + " | " + status
    );
  }

  if (total > 0) {
    double percentage = present * 100.0 / total;
    System.out.printf("Attendance: %.2f%%\n",
    percentage);
  } else {
    System.out.println("No Attendance Records Found.");
  }

} catch (Exception e) {
```

```
  }
}
static void
  viewAcademic(int id) {
  String sql =
    "SELECT * FROM academic_details
  WHERE student_id=?"; try (Connection
  con = DBConnection.getConnection()) {
    PreparedStatement ps =
    con.prepareStatement(sql);
    ps.setInt(1, id);

    ResultSet rs =
    ps.executeQuery(); if
    (rs.next()) {
      System.out.println("\n========== ACADEMIC DETAILS ==========");
      System.out.println("Admission Year: " +
      rs.getInt("admission_year"));
      System.out.println("Course: " + rs.getString("course"));
      System.out.println("Department: " +
      rs.getString("department")); System.out.println("Year: "
      + rs.getInt("current_year"));
      System.out.println("Semester: " +
      rs.getInt("current_semester"));
      System.out.println("CGPA: " + rs.getBigDecimal("cgpa"));
    } else {
      System.out.println("Academic Details Not Found.");
    }
```

```
  } catch (Exception e) {
    System.out.println(e.getMessa
    ge());
  }
}

static void changePassword(int id) {
  try (Connection con =
    DBConnection.getConnection()) {
    System.out.print("Current Password:
    ");
    String current = Main.sc.nextLine();

    String sql =
      "SELECT password_hash FROM users WHERE student_id=? AND role='STUDENT'";

    PreparedStatement ps =
    con.prepareStatement(sql);
    ps.setInt(1, id);
    ResultSet rs =
    ps.executeQuery(); if
    (!rs.next()) {
      System.out.println("Account Not Found.");
```

```
}

if (!PasswordUtil.verifyPassword(
  current, rs.getString("password_hash"))) {

  System.out.println("Incorrect
  Current Password."); return;
}

System.out.print("New
Password: "); String
newPassword =
Main.sc.nextLine();

System.out.print("Confirm
Password: "); String
confirm =
Main.sc.nextLine();

if (newPassword.length() < 8) {
  System.out.println("Password must contain
  at least 8 characters."); return;
}

if (!newPassword.equals(confirm)) {
  System.out.println("Passwords Do
  Not Match."); return;
```

```
}

String update =
  "UPDATE users SET password_hash=? WHERE
student_id=? AND role='STUDENT'"; PreparedStatement
ups = con.prepareStatement(update);
ups.setString(1,
```

```
    ps.setInt(4,
    obtained);
    ps.setString(5,
    exam);
    ps.setDate(6,
    java.sql.Date.valueOf(date));
    ps.executeUpdate();
    System.out.println("Marks Added
    Successfully!"); ps.close();
  } catch (Exception e) {
    System.out.println("Error: " +
    e.getMessage());
  }
}

static void addAttendance() {
  try (Connection con = DBConnection.getConnection()) {

    System.out.print("Enter
    Student ID: "); int id =
    Main.readInt();

    System.out.print("Enter
    Subject: "); String
    subject =
    Main.sc.nextLine();
```

```
System.out.print("Enter Date
(YYYY-MM-DD): "); String date =
Main.sc.nextLine();

System.out.println("\n===== ATTENDANCE
STATUS ====="); System.out.println("1.
PRESENT");
System.out.println("2. ABSENT");

System.out.print("Enter
Choice (1 or 2): "); int choice
= Main.readInt();

String status;

if (choice == 1)
  { status =
  "PRESENT";
} else if (choice
  == 2) { status =
  "ABSENT";
} else {
  System.out.println("Invalid Choice!
  Enter 1 or 2."); return;
}

String sql = "INSERT INTO
```

```
attendance(student_id,subject,attendance_date,status) VALUES(?,?,?,?)";
PreparedStatement ps = con.prepareStatement(sql);


ps.setInt(1, id);
ps.setString(2,
subject);
ps.setDate(3, java.sql.Date.valueOf(date));
```

```
ps.setString(4,
status);
ps.executeUpd
ate();
System.out.println("Attendance Added
Successfully!");
```

5.4 STUDENT DASHBOARD

After successful student login, StudentDashboard.show(studentId) displays the student menu. All student queries use the authenticated student ID, which prevents the student menu from displaying another student's records through ordinary dashboard operations. Attendance percentage is calculated as present_count × 100 / total_attendance_records.

STUDENT DASHBOARD CODE (StudentDashboard.java)

```
import java.sql.*;


public class StudentDashboard {


  public static void show(int
    studentId) { while (true) {
      System.out.println("\n========== STUDENT DASHBOARD ==========");
      System.out.println("1. View My
      Profile"); System.out.println("2. View
      My Marks"); System.out.println("3.
      View My Attendance");
      System.out.println("4. View Academic
      Details"); System.out.println("5.
      Change Password");
      System.out.println("6. Logout");
      System.out.print("Ente
      r choice: "); int choice =
      Main.readInt(); switch
      (choice) {
        case 1: viewProfile(studentId); break;
        case 2: viewMarks(studentId); break;
        case 3:
```

```
        viewAttendance(studentId);
        break; case 4:
        viewAcademic(studentId);
        break; case 5:
        changePassword(studentId);
        break; case 6:
          System.out.println("Student
          Logged Out."); return;
        default:
          System.out.println("Invalid Choice.");
      }
    }
}

static void viewProfile(int id) {

  String sql = "SELECT * FROM students
  WHERE student_id=?"; try (Connection
  con = DBConnection.getConnection()) {
    PreparedStatement ps =
    con.prepareStatement(sql);
    ps.setInt(1, id);
```

```
  ResultSet rs =
  ps.executeQuery(); if
  (rs.next()) {
    System.out.println("\n========== MY
    PROFILE ==========");
    System.out.println("ID: " +
    rs.getInt("student_id"));
    System.out.println("Name: " +
    rs.getString("name"));
    System.out.println("Age: " + rs.getInt("age"));
    System.out.println("Department: " +
    rs.getString("department"));
    System.out.println("Email: " +
    rs.getString("email"));
    System.out.println("Phone: " +
    rs.getString("phone"));
    System.out.println("Course: " +
    rs.getString("course"));
    System.out.println("Year: " +
    rs.getInt("year"));
    System.out.println("Semester: " +
    rs.getInt("semester"));
  } else {
    System.out.println("Profile Not Found.");
  }


} catch (Exception e) {
```

```
    System.out.println(e.getMessa
    ge());
  }
}

static void viewMarks(int id) {

  String sql = "SELECT * FROM marks
  WHERE student_id=?"; try (Connection
  con = DBConnection.getConnection()) {
    PreparedStatement ps =
    con.prepareStatement(sql);
    ps.setInt(1, id);

    ResultSet rs = ps.executeQuery();
    System.out.println("\n========== MY
    MARKS ==========="); boolean found =
    false;
    while
      (rs.next()) {
      found = true;
      System.out.println("Subject: " +
      rs.getString("subject"));
      System.out.println("Exam: " +
      rs.getString("exam_type"));
      System.out.println("Total: " +
      rs.getInt("total_marks"));
```

```
  System.out.println("Obtained: " +
  rs.getInt("obtained_marks"));
  System.out.println("      ");
}
```

```
        System.out.println("No marks found.");
      }


    } catch (Exception e) {
      System.out.println(e.getMessa
      ge());
    }
}
static void
  viewAttendance(int id)
  { String sql =
    "SELECT * FROM attendance WHERE student_id=? ORDER
  BY attendance_date DESC"; try (Connection con =
  DBConnection.getConnection()) {
    PreparedStatement ps =
    con.prepareStatement(sql);
    ps.setInt(1, id);
    ResultSet rs =
    ps.executeQuery(); int
    total = 0;
    int present = 0;


    System.out.println("\n========== MY ATTENDANCE ==========");


    while
      (rs.next()) {
      total++;
```

```
    String status = rs.getString("status");

    if
      (status.equals("PRES
      ENT")) { present++;
    }

    System.out.println(
      rs.getString("subject") + "
      | " +
      rs.getDate("attendance_
      date") + " | " + status
    );
  }

  if (total > 0) {
    double percentage = present * 100.0 / total;
    System.out.printf("Attendance: %.2f%%\n",
    percentage);
  } else {
    System.out.println("No Attendance Records Found.");
  }

} catch (Exception e) {
```

```
  }
}
static void
  viewAcademic(int id) {
  String sql =
    "SELECT * FROM academic_details
  WHERE student_id=?"; try (Connection
  con = DBConnection.getConnection()) {
    PreparedStatement ps =
    con.prepareStatement(sql);
    ps.setInt(1, id);

    ResultSet rs =
    ps.executeQuery(); if
    (rs.next()) {
      System.out.println("\n========== ACADEMIC DETAILS ==========");
      System.out.println("Admission Year: " +
      rs.getInt("admission_year"));
      System.out.println("Course: " + rs.getString("course"));
      System.out.println("Department: " +
      rs.getString("department")); System.out.println("Year: "
      + rs.getInt("current_year"));
      System.out.println("Semester: " +
      rs.getInt("current_semester"));
      System.out.println("CGPA: " + rs.getBigDecimal("cgpa"));
    } else {
      System.out.println("Academic Details Not Found.");
    }
```

```
  } catch (Exception e) {
    System.out.println(e.getMessa
    ge());
  }
}

static void changePassword(int id) {
  try (Connection con =
    DBConnection.getConnection()) {
    System.out.print("Current Password:
    ");
    String current = Main.sc.nextLine();

    String sql =
      "SELECT password_hash FROM users WHERE student_id=? AND role='STUDENT'";

    PreparedStatement ps =
    con.prepareStatement(sql);
    ps.setInt(1, id);
    ResultSet rs =
    ps.executeQuery(); if
    (!rs.next()) {
      System.out.println("Account Not Found.");
```

```
}

if (!PasswordUtil.verifyPassword(
  current, rs.getString("password_hash"))) {

  System.out.println("Incorrect
  Current Password."); return;
}

System.out.print("New
Password: "); String
newPassword =
Main.sc.nextLine();

System.out.print("Confirm
Password: "); String
confirm =
Main.sc.nextLine();

if (newPassword.length() < 8) {
  System.out.println("Password must contain
  at least 8 characters."); return;
}

if (!newPassword.equals(confirm)) {
  System.out.println("Passwords Do
  Not Match."); return;
```

```
}

String update =
  "UPDATE users SET password_hash=? WHERE
student_id=? AND role='STUDENT'"; PreparedStatement
ups = con.prepareStatement(update);
ups.setString(1,
```

5.5 DATABASE TABLES

The database name used by the application is student_management. The tables shown below reflect the fields accessed by the Java code and the supplied database screenshots. Primary and foreign key constraints should be configured in MySQL to preserve referential integrity.

STUDENTS TABLE

Column	Description
student_id	Primary key / auto-increment identifier
name	Student full name
age	Student age
department	Department name
email	Email address
phone	Contact number
course	Course name
year	Current academic year
semester	Current semester
created_at	Record creation timestamp

USERS TABLE

Column	Description
user_id	Primary key
username	Unique login username
password_hash	Salted PBKDF2 password hash
role	ADMIN or STUDENT
student_id	Student reference for student accounts
account_status	ACTIVE or INACTIVE

ACADEMIC_DETAILS TABLE

Column	Description
academic_id	Primary key
student_id	Student reference
admission_year	Admission year
course	Course

department	Department
current_year	Current year
current_semester	Current semester
cgpa	Grade point average (optional)

MARKS TABLE

Column	**Description**
mark_id	Primary key
student_id	Student reference
subject	Subject name
total_marks	Maximum marks
obtained_marks	Marks obtained
exam_type	Exam category
exam_date	Exam date

ATTENDANCE TABLE

Column	Description
attendance_id	Primary key
student_id	Student reference
subject	Subject name
attendance_date	Date
status	PRESENT or ABSENT

5.6 DATABASE SCHEMA (SQL)

```
CREATE DATABASE IF NOT EXISTS
student_management; USE
student_management;

CREATE TABLE students (
 student_id INT
 AUTO_INCREMENT PRIMARY KEY,
 name VARCHAR(100) NOT NULL,
 age INT,
 department
 VARCHAR(100),
 email
 VARCHAR(120),
 phone
 VARCHAR(20),
 course
 VARCHAR(80),
 year INT,
 semester INT,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE users (
 user_id INT AUTO_INCREMENT
 PRIMARY KEY, username
 VARCHAR(80) UNIQUE NOT NULL,
```

```
 password_hash VARCHAR(255)
 NOT NULL, role VARCHAR(15)
 NOT NULL,
 student_id INT NULL,
 account_status VARCHAR(15) DEFAULT 'ACTIVE',
 FOREIGN KEY (student_id) REFERENCES
  students(student_id) ON DELETE
  CASCADE
);

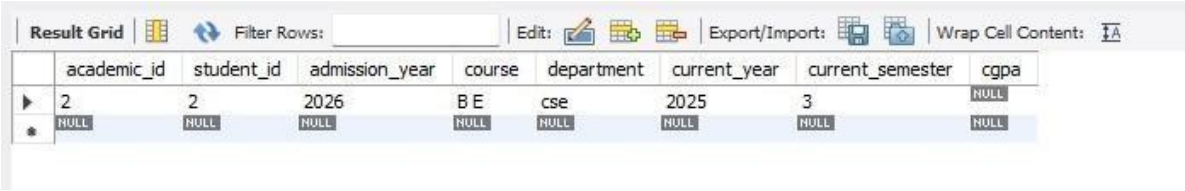
CREATE TABLE academic_details (
 academic_id INT
 AUTO_INCREMENT PRIMARY KEY,
 student_id INT UNIQUE NOT NULL,
 admission_year
 INT, course
 VARCHAR(80),
 department
 VARCHAR(100),
 current_year INT,
 current_semester
 INT,
 cgpa DECIMAL(4,2),
 FOREIGN KEY (student_id) REFERENCES
  students(student_id) ON DELETE
  CASCADE
);
```

```
CREATE TABLE marks (
 mark_id INT AUTO_INCREMENT PRIMARY KEY,
 student_id INT
 NOT NULL, subject
 VARCHAR(100),
 total_marks INT,
 obtained_marks
 INT, exam_type
 VARCHAR(50),
 exam_date DATE,
 FOREIGN KEY (student_id) REFERENCES
  students(student_id) ON DELETE
  CASCADE
);
```

```
CREATE TABLE attendance (
 attendance_id                    INT
 AUTO_INCREMENT   PRIMARY   KEY,
 student_id INT NOT NULL,
 subject
 VARCHAR(100),
 attendance_dat
 e  DATE,  status
 VARCHAR(10),
 FOREIGN  KEY  (student_id)  REFERENCES
  students(student_id)      ON      DELETE
  CASCADE
);
```

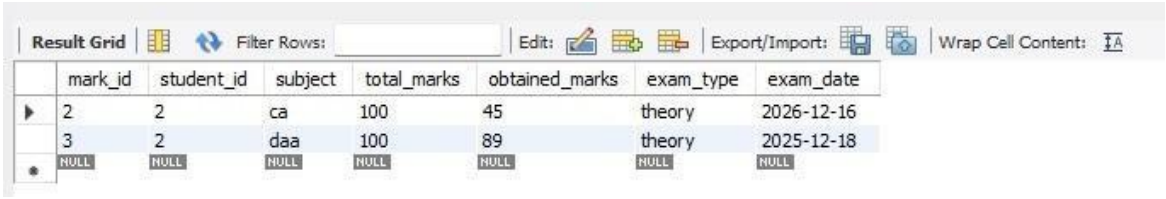
Note: The above SQL is a report-ready schema representation based on the Java fields and screenshots. Confirm column definitions and foreign keys against the exact MySQL script used during execution before final submission.

5.7 SAMPLE DATABASE OUTPUTS



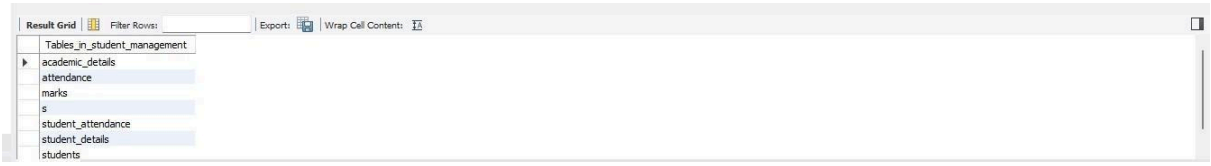
academic_id	student_id	admission_year	course	department	current_year	current_semester	cgpa
2	2	2026	B E	cse	2025	3	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 5.6(a): Student record displayed in MySQL result grid



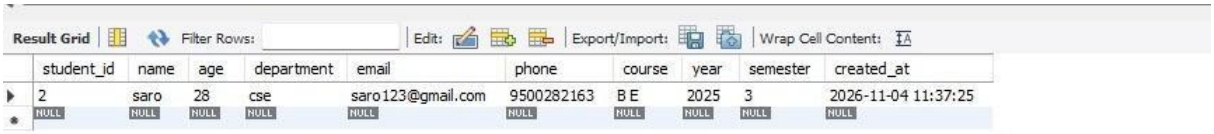
mark_id	student_id	subject	total_marks	obtained_marks	exam_type	exam_date
2	2	ca	100	45	theory	2026-12-16
3	2	daa	100	89	theory	2025-12-18
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 5.6(b): Academic details table result



Tables_in_student_management
academic_details
attendance
marks
s
student_attendance
student_details
students

Figure 5.6(c): Marks table result



student_id	name	age	department	email	phone	course	year	semester	created_at
2	saro	28	cse	saro123@gmail.com	9500282163	B E	2025	3	2026-11-04 11:37:25
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 5.6(d): Database table listing

10· VERIFICATION & TEST PLAN

The following test cases can be used to verify the major operations. Actual test execution should be recorded with the observed result before submission.

Test ID	Test Scenario	Expected Result
TC01	Login with valid admin credentials	Admin dashboard opens.
TC02	Login with incorrect password	Login is rejected with an error message.
TC03	Login using wrong role	Access denied.
TC04	Add a student with valid details	Student record, account and academic record are saved.
TC05	Add student with duplicate username	Database constraint/error prevents duplicate account.
TC06	Search student by existing ID	Matching student details are displayed.
TC07	Search student by name	Matching name records are displayed.
TC08	Update student information	Selected fields are updated successfully.
TC09	Delete student after confirmation	Student record is removed; related rows depend on FK cascade.
TC10	Enter obtained marks greater than total	Validation rejects the marks.
TC11	Record PRESENT attendance	Attendance row is inserted.
TC12	Student views marks	Only logged-in student's marks are displayed.
TC13	Student views attendance	History and attendance percentage are displayed.
TC14	Change password with wrong current password	Password change is rejected.

TC15	Change password with valid inputs	New password hash is saved.
TC16	Login with inactive student account	Login is rejected.

10.1 VALIDATION

- Numeric menu input is validated using readInt() and NumberFormatException handling.
- Marks validation checks total marks > 0 and 0 ≤ obtained marks ≤ total marks.
- Password change checks minimum length and confirmation match.
- Student login verifies password and role and checks account status.
- Student creation uses a transaction and rollback on failu

11 · CONCLUSION & FUTURE WORK

The Student Management System provides a centralized and structured approach to student record management using Java and MySQL. It reduces the effort involved in maintaining student details manually and enables administrators to manage records, accounts, marks and attendance through a menu-driven application. Students can securely access their own profile, academic performance and attendance information.

The project demonstrates Java programming, classes and objects, exception handling, JDBC connectivity, SQL CRUD operations, relational database design, joins, transactions and password hashing. Future enhancements may include a graphical interface, automated reports, notifications, advanced analytics and deployment in an institutional environment.

12· REFERENCES

- Java Platform Documentation – https://docs.oracle.com/en/java/
- MySQL Documentation – https://dev.mysql.com/doc/
- MySQL Connector/J Documentation – https://dev.mysql.com/doc/connector-j/en/
- Java JDBC API Documentation – https://docs.oracle.com/javase/
- Course material: CS23333 – Object Oriented Programming Using Java and Database Management Systems.